



Tracking FSL

Bottitta Marco, Bivona Francesco e
Strazzanti Pierluigi

5 A INF - 2026



TRACKING-FSL

Di che cosa tratta questo progetto?

Il progetto realizzato consiste in un sistema web per la gestione delle **FSL** (gestione utenti, tutor e studenti) sviluppato tramite tecnologie web sia lato client che lato server.

L'applicazione permette l'autenticazione di diversi tipi di utenti (Admin, Tutor e Studenti) attraverso un sistema di login collegato ad un database relazionale.



Tecnologie utilizzate

Il sito è stato sviluppato utilizzando diverse tecnologie come:



HTML → per quanto struttura delle pagine web



CSS → per migliorare lo stile grafico e layout



PHP → riguardo la logica lato server e gestione dati



SQL/MySQL → per la gestione del database

```
<!DOCTYPE html>

<html lang="en" xmlns="http://www.w3.org/1999/xhtml">
<head>
  <meta charset="utf-8" />
  <title>Numeric Input Form</title>
  <style>
    #main {
      font-family: Arial, Verdana, sans-serif;
    }

    h1 {
      color: blue;
      text-align: center;
    }
  </style>
</head>
<body id="main">
  <h1>Numeric Input</h1>
  <form>
    <label for="radius">Radius (units)</label>
    <p></p>
    <input style="width: 20%;" name="radius" id="radius" placeholder="Enter
      radius (0 to 100 units)"/>
    <p></p>
    <input type="button" onclick="calculateAreaOfCircle()" value="Calculate
      Area of Circle">

    <p id="display"></p>

  </form>

  <script>
    function calculateAreaOfCircle() {
      var txt = "";
      var r;
      var a;

      r = document.getElementById("radius").value;
      if (isNaN(r) || r < 0 || r > 100) {
        txt = "Invalid input. Must be a number between 0 and 100
          units.";
      } else {
        a = Math.PI * r * r;
        txt += "You have entered " + r + " units" + "<br>";
        txt += "Area of circle is " + a.toFixed(2) + " square units";
      }
      document.getElementById("display").style.color = "green";
      document.getElementById("display").innerHTML = txt;
    }

  </script>

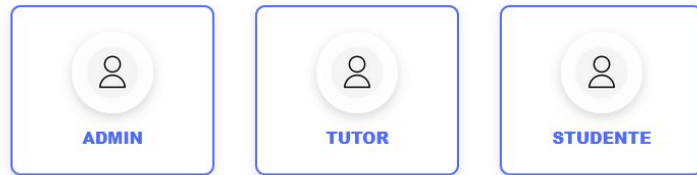
</body>
</html>
```



Obiettivo principale

L'obiettivo principale del progetto è creare un ambiente in cui l'amministratore possa gestire i tutor i quali possano in seguito gestire gli studenti permettendogli l'accesso alla piattaforma e visionare la propria dashboard.

Scegli un ruolo



Cos'è un database?

Un **Database** è un insieme organizzato di dati memorizzati elettronicamente, utilizzato per salvare, modificare e recuperare informazioni in modo rapido ed efficiente.

Nel progetto è stato utilizzato un database relazionale **MySQL**, che organizza i dati in tabelle composte da: righe (record), colonne (campi)

Esempi di tabelle presenti nel progetto: tutor, studente

Ogni tabella contiene informazioni specifiche sugli utenti: username, password, ID utente e privilegi admin

Cos'è un database e a cosa serve

Un database è un sistema che consente di raccogliere, organizzare e gestire grandi quantità di dati in modo efficiente.



E-commerce



Gestisce gli ordini e l'inventario dei prodotti

Sanità



Memorizza le cartelle cliniche dei pazienti

Scuola



Archivia i dati

Social Media



Organizza gli utenti



Tecnologie Utilizzate

HTML

L'HTML (**HyperText Markup Language**) è il linguaggio utilizzato per creare la struttura delle pagine web ed è alla base di ogni singola pagina web esistente al mondo

Nel progetto HTML viene usato per: creare form di login, creare dashboard, creare pagine di registrazione ed infine inserire pulsanti e campi input.

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>Home Page</title>
  <link rel="stylesheet" href="HomePage.css">
</head>

<body style="padding-top: 78px;">
  <iframe src="Header.html" title="Header"
    style="width: 100%; border: 0; height: 78px; display: block; position: fixed; top: 0; left: 0; z-index: 9999;"
    scrolling="no"></iframe>
  <h1>Scegli un ruolo</h1>
  <div class="forms-container">
    <form action="login.php" method="post" onclick="this.submit()">
      <input type="hidden" name="role" value="admin">
      
      <b>ADMIN</b>
    </form>
    <form action="login.php" method="post" onclick="this.submit()">
      <input type="hidden" name="role" value="tutor">
      
      <b>TUTOR</b>
    </form>
    <form action="login.php" method="post" onclick="this.submit()">
      <input type="hidden" name="role" value="studente">
      
      <b>STUDENTE</b>
    </form>
  </div>
  <iframe src="Footer.html" title="Footer"
    style="width: 100%; border: 0; height: 90px; margin-top: auto; display: block;" scrolling="no"></iframe>
</body>

</html>
```



CSS

CSS (**C**ascading **S**tyle **S**heets) serve per definire l'aspetto grafico del sito.

Con il CSS è possibile: modificare colori, gestire dimensioni, creare layout responsive, migliorare l'interfaccia utente.

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  margin: 0;
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: flex-start;
  background-color: #f5f9ff;
  font-family: Arial, sans-serif;
  width: 100%;
}

.page-content {
  flex: 1;
  width: 100%;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 24px 16px;
}

.login-box {
  width: 420px;
  padding: 30px 36px;
  background-color: #ffffff;
  border-radius: 10px;
  border: 2px solid #007bff;
  box-shadow: 0 4px 10px rgba(0, 0, 0, 0.1);
}

.login-box h1 {
  text-align: center;
  color: #007bff;
  margin-bottom: 28px;
  font-size: 2rem;
}

.login-box form {
  display: flex;
  flex-direction: column;
  gap: 22px;
}
```



PHP

PHP è il linguaggio lato server utilizzato per:

- gestire il login;
- controllare le sessioni;
- comunicare con il database;
- eseguire query SQL;
- gestire utenti e autenticazione.

Nel progetto PHP rappresenta il collegamento principale tra:

- frontend (HTML/CSS)
- database MySQL

```
k?php
    $mysql_host = "localhost";
    $mysql_user = "root";
    $mysql_pass = "";
    $mysql_db = "TrackingFSL";
    $conn = mysqli_connect($mysql_host, $mysql_user, $mysql_pass, $mysql_db);
    if(!$conn){
        die("Connection failed: " . mysqli_connect_error());
    }
?>
```



Funzioni principali del PHP



session_start()

Avvia una sessione PHP.

Le sessioni servono per mantenere l'utente autenticato durante la navigazione del sito.

```
<?php
session_start();
include 'config.php';
if(isset($_SESSION['loggatoAdmin'])||isset($_SESSION['loggatoTutor'])||isset($_SESSION['loggatoStudiante']))){
    if($_SESSION['loggatoAdmin']){
        header("Location: registrazioneTutor.html");
        exit;
    }else if($_SESSION['loggatoTutor']){
        header("Location: registrazioneStudiante.html");
        exit;
    }else if($_SESSION['loggatoStudiante']){
        header("Location: dashboardStudiante.html");
        exit;
    }
}else{
    header("Location: login.html");
    $username1 = $_POST['username'];
    $password = $_POST['password'];
    $sql = "SELECT username FROM tutor WHERE username='$username1'";
    $sql2 = "SELECT username FROM studente WHERE username='$username1'";
    $result = mysqli_query($conn, $sql);
    $result2 = mysqli_query($conn, $sql2);
    if($result->num_rows > 0){
        $sql3 = "SELECT access_password FROM tutor WHERE access_password='admin'";
        $result3 = mysqli_query($conn, $sql3);
        if($result3->num_rows > 0){
            $row = mysqli_fetch_assoc($result3);
            if($row['access_password'] == $password){
                $_SESSION['loggatoAdmin'] = true;
                header("Location: registrazioneTutor.html");
                echo('<a href="login.html" target="_top" name="login">$username1</a>');
                exit;
            }
        }
    }
}
```



include()

```
include 'config.php';  
session_start();
```

L'istruzione include in PHP serve a inserire il contenuto di un file esterno (come header, footer o menu) all'interno di un'altra pagina PHP prima che venga eseguita, agendo come un "copia e incolla" automatico. È fondamentale per la modularità, consentendo di riutilizzare codice, aggiornare più pagine modificandone una sola e mantenere il sito organizzato.

```
<?php  
include 'config.php';  
session_start();  
$username = $_POST['username'];  
$password = $_POST['password'];  
$sql="UPDATE tutor SET is_admin=true WHERE id_tutor=658974";  
if (isset($sql, $_SESSION['loggato']) && $_SESSION['loggato'] === true) {  
    echo('<script type="text/javascript">alert("SEI GIA LOGGATO");</script>');  
    header('Location: registrazioneTutor.html');  
    exit;  
}else{  
    header('Location: login.html');  
    if ($username === 'ADMIN' && $password === 'admin') {  
        echo('<script type="text/javascript">alert("ACCESSO EFFETTUATO CON SUCCESSO");</script>');  
        $_SESSION['loggato'] = true;  
    } else {  
        echo('<script type="text/javascript">alert("USERNAME O PASSWORD ERRATI");</script>');  
    }  
}  
?>
```



\$_POST

```
$username1 = $_POST['username'];  
$password = $_POST['password'];
```

Recupera i dati inviati da un form HTML tramite metodo POST.

Serve per leggere l'username e la password inseriti dall'utente.

Il metodo POST viene utilizzato principalmente per inviare dati sensibili o voluminosi a un server web (come moduli, login o file), garantendo maggiore sicurezza e privacy rispetto a GET.

```
<?php  
include 'config.php';  
session_start();  
$username = $_POST['username'];  
$password = $_POST['password'];  
$sql="UPDATE tutor SET is_admin=true WHERE id_tutor=658974";  
if (isset($sql, $_SESSION['loggato']) && $_SESSION['loggato'] === true) {  
    echo('<script type="text/javascript">alert("SEI GIA LOGGATO");</script>');  
    header('Location: registrazioneTutor.html');  
    exit;  
}  
else{  
    header('Location: login.html');  
    if ($username === 'ADMIN' && $password === 'admin') {  
        echo('<script type="text/javascript">alert("ACCESSO EFFETTUATO CON SUCCESSO");</script>');  
        $_SESSION['loggato'] = true;  
    }  
    else {  
        echo('<script type="text/javascript">alert("USERNAME O PASSWORD ERRATI");</script>');  
    }  
}  
?>
```



Connessione al Database

La connessione avviene tramite **\$conn** che contiene la connessione MySQL definita nel file **config.php**

```
<?php
    $mysql_host = "localhost";
    $mysql_user = "root";
    $mysql_pass = "";
    $mysql_db = "TrackingFSL";
    $conn = mysqli_connect($mysql_host, $mysql_user, $mysql_pass, $mysql_db);
    if(!$conn){
        die("Connection failed: " . mysqli_connect_error());
    }
?>
```



Funzioni PHP per SQL

mysqli_query()

Esegue una query SQL sul database.

Utilizza due parametri principali:

`$conn` → connessione database; `$sql` → query da eseguire

È una delle funzioni più importanti del progetto.

```
*abc.php
1 <?php
2 //Establishment of Connection with database
3 $connection = mysqli_connect("localhost","maadi","qwer1234","Organization");
4
5 //checking status of connection
6 if (mysqli_connect_errno()) {
7     echo "Failed to connect to MySQL: " . mysqli_connect_error();
8     exit();
9 }
10 //declaring variable
11 $firstname2 = "John'o Alexander";
12
13 //pass from the mysqli_real_escape_string()
14 $firstname = mysqli_real_escape_string($connection, $firstname2);
15 //inserting into the database
16 $sql="INSERT INTO Linuxhint (Emp_name) VALUES ('$firstname')";
17
18 //execution of the mysql queries
19 $r = mysqli_query($connection, "INSERT into Linuxhint VALUES ('$firstname')");
20 if(!$r){
21     print("Error occurred\n");
22 }
23 else{
24     print("Record inserted successfully\n");
25 }
26 //connection closed
27 mysqli_close($connection);
28 ?>
```

num_rows

Controlla se la query ha trovato risultati.

Se il numero di righe è maggiore di 0 significa che l'utente esiste nel database.

```
765     $retval = array();
766     $query = $this->query($sql);
767
768     if ($query->num_rows() > 0)
769     {
770         foreach ($query->result_array() as $row)
771         {
772             if (isset($row['TABLE_NAME']))
773             {
774                 $retval[] = $row['TABLE_NAME'];
775             }
776             else
777             {
778                 $retval[] = array_shift($row);
779             }
780         }
781     }
782
783     $this->data_cache['table_names'] = $retval;
784     $ex=$this->data_cache['table_names'];
```



Funzioni del sito

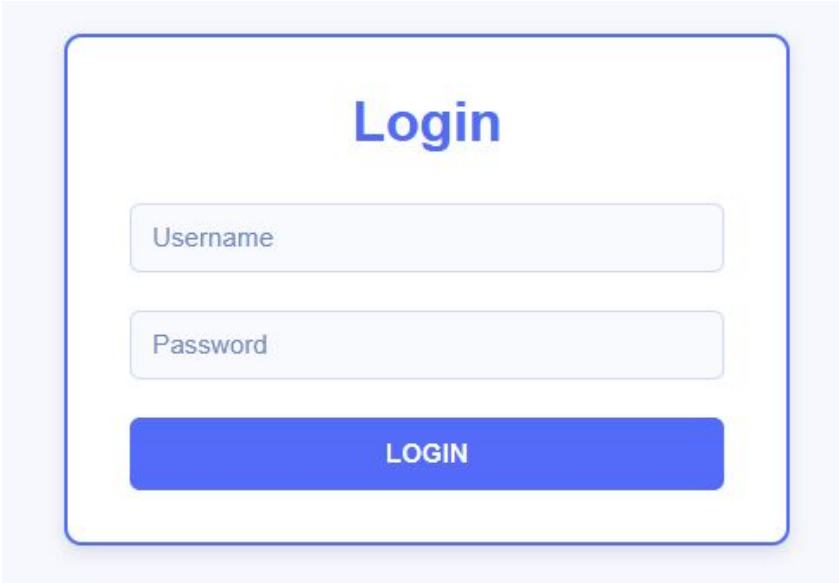


Gestione Login

Nel progetto il login funziona in questo modo:

1. L'utente inserisce username e password.
2. PHP legge i dati tramite \$_POST.
3. Viene eseguita una query SELECT.
4. Se i dati sono corretti:
 - viene creata una sessione;
 - l'utente viene reindirizzato alla propria pagina.

Questa istruzione salva la sessione dell'amministratore.



A login form UI mockup with a light blue background. The form is enclosed in a rounded rectangle with a blue border. At the top, the word "Login" is written in a large, bold, blue font. Below it, there are two input fields: the first is labeled "Username" and the second is labeled "Password", both in a light blue font. At the bottom of the form, there is a solid blue button with the word "LOGIN" in white, uppercase letters.



Registrazione Tutor

Una volta entrati come Admin potremo registrare un Tutor.

Qui si avvia la sessione e verifica se il tutor è regolarmente loggato.

Esegue una query SQL per prelevare dal database tutte le informazioni personali del tutor associate al suo username.

Esegue una seconda query SQL per recuperare tutte le attività di FSL create da quello specifico tutor.

Registrazione Tutor

Nome:

Cognome:

Data di nascita:



Luogo di nascita:

Sesso



Email:

Scuola:

Materie:

Titolo di studio:

Username:

Password:

Registrati

Dashboard Tutor

Qui si potranno inserire le attività che vogliamo.

I dati personali e i campi del form si basano sulle query e le tabelle gestite sul database dal back-end in PHP.



Marco Rossi
Admin
m.rossi@example.com

Dashboard Tutor

Inserisci nuova attività FSL

Compila il modulo per aggiungere un'attività.

Titolo

Descrizione

Data inizio

gg / mm / aaaa

Data fine

gg / mm / aaaa

Ore previste

Modalità

☒ presenza
☐ online

Azienda

Convenzione

Salva attività

Attività assegnate

Elenco delle attività

Errore nel caricamento delle attività.



Dashboard Tutor

Qui vedremo tutte le attività salvate
precedentemente inerenti a un determinato tutor

Dashboard Tutor



Tutor
Materia
Email

Le mie attività FSL

Elenco delle attività che hai creato per i tuoi studenti

Non hai creato nessuna attività ancora.

Data: - | Ore previste: | Ore fatte:
Modalità: | Convenzione:



Gestione Password



Gestione Password

Nel file config.php **criptaPassword** prende la password in chiaro e la trasforma in un hash con l'algoritmo **bcrypt**. L'hash è una stringa irriconoscibile e non reversibile — non è possibile risalire alla password originale. Ogni volta che viene chiamata, genera un hash leggermente diverso, ma tutti restano validi.

verificaPassword non decripta nulla: confronta la password inserita al login con l'hash salvato nel database, restituendo true o false.

```
function criptaPassword($password) {  
    return password_hash($password, PASSWORD_BCRYPT);  
}  
  
/**  
 * Verifica una password confrontandola con l'hash memorizzato  
 * @param string $password Password in chiaro da verificare  
 * @param string $hash Hash memorizzato nel database  
 * @return bool True se la password è corretta, False altrimenti  
 */  
function verificaPassword($password, $hash) {  
    return password_verify($password, $hash);  
}
```

?>



Gestione Password

Il file registrazioneStudente.php si divide in due blocchi logici:

Se l'utente è già loggato, viene reindirizzato alla sua pagina. Solo chi **non è loggato** arriva al blocco di registrazione.

Raccoglie tutti i campi dal form
Cripa la password con `criptaPassword()` (bcrypt)
Salva nel DB **solo l'hash**, mai la password in chiaro

```
if(isset($_SESSION['loggatoAdmin'])||isset($_SESSION['loggatoTutor'])||isset($_SESSION['loggatoStudente'])){\n    if($_SESSION['loggatoAdmin']){\n        header("Location: registrazioneTutor.html");\n        exit;\n    }else if($_SESSION['loggatoTutor']){\n        header("Location: registrazioneStudente.html");\n        exit;\n    }else if($_SESSION['loggatoStudente']){\n        header("Location: dashboardStudente.html");\n        exit;\n    }\n}
```

```
$nome=$_POST['nome'];\n$congnome=$_POST['cognome'];\n$data_nascita=$_POST['data_nascita'];\n$luogo_nascita=$_POST['luogo_nascita'];\n$ sesso=$_POST['sesso'];\n$email=$_POST['email'];\n$scuola=$_POST['scuola'];\n$materia=$_POST['materie'];\n$titolo_studio=$_POST['titolo_studio'];\n$username=$_POST['username'];\n$password=$_POST['password'];
```



Gestione Password

Il login.php gestisce invece tre fasi: logout, utente già loggato, login vero e proprio e login vero e proprio

Caso 1 — Logout:

```
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['role'])) {  
    session_unset(); // Rimuove tutte le variabili di sessione  
    session_destroy(); // Distrugge la sessione attiva  
    header("Location: login.html");  
    exit;  
}
```

Caso 2 — Già loggato:

```
if (isset($_SESSION['loggatoAdmin']) && $_SESSION['loggatoAdmin'] === true) {  
    header("Location: registrazioneTutor.html");  
    exit;  
} elseif (isset($_SESSION['loggatoTutor']) && $_SESSION['loggatoTutor'] === true) {  
    header("Location: registrazioneStudente.html");  
    exit;  
} elseif (isset($_SESSION['loggatoStudente']) && $_SESSION['loggatoStudente'] === true) {  
    header("Location: dashboardStudente.html");  
    exit;  
}
```

Caso 3 — Login vero e proprio (tre sotto-casi):

Admin —► confronto diretto in chiaro
(username === 'ADMIN' && password === 'admin') | Tutor —► query DB →
verificaPassword(\$password, \$hash_dal_db)
| Studente —► query DB →
verificaPassword(\$password, \$hash_dal_db)



Utilizzo JavaScript



JS in HomePage.html

```
<form action="login.php" method="post" onclick="this.submit()">
```

Non è un `<script>` vero e proprio, ma un **event handler inline**. Il `onclick="this.submit()"` fa sì che cliccando *ovunque* sul form (sull'immagine, sul testo) questo venga subito inviato, senza bisogno di un bottone dedicato. Ogni form invia un campo nascosto `role` con il valore corrispondente (admin, tutor, studente).



JS in RegistrazioneStudiante.html

Aspetta che la pagina sia completamente caricata (DOMContentLoaded), poi chiama `tutorDinamici.php` in background tramite `fetch`. Il PHP risponde con una serie di `<option>` HTML che vengono iniettate nella `<select id="id_tutor">`. In questo modo la lista dei tutor disponibili è **sempre aggiornata dal database**, senza ricaricare la pagina.

```
<script>
  document.addEventListener("DOMContentLoaded", function() {
    const selectTutor = document.getElementById("id_tutor");

    // Chiama il file PHP in background
    fetch("tutorDinamici.php")
      .then(response => response.text())
      .then(htmlGenerato => {
        // Aggiunge le <option> stampate dal PHP dentro la select
        selectTutor.innerHTML += htmlGenerato;
      })
      .catch(error => {
        console.error("Errore nel caricamento dei tutor:", error);
      });
  });
</script>
```



JS in DashboardStudiante.html

Il codice fa **una sola cosa**: chiedere dati al server e mostrarli nella pagina, senza ricaricarla. Lo fa due volte, una per il nome dello studente e una per le attività.

```
<script>
// 1. CARICA I DATI DELLO STUDENTE
fetch("caricaDatiStudiante.php")
  .then(response => response.text())
  .then(htmlGenerato => {
    // Sostituisce il placeholder con i dati reali dello studente
    document.getElementById("studente-name").innerHTML = htmlGenerato;
  })
  .catch(error => {
    console.error("Errore nel caricamento del profilo:", error);
    profileInfo.innerHTML = '<h2 class="studente-name">Errore nel caricamento.</h2>';
  });

// Recuperiamo l'elemento corretto usando il suo ID "fsl-activities"
const listActivities = document.getElementById("fsl-activities");
//Carica la lista delle attività
fetch("caricaAttivitaStudiante.php")
  .then(response => response.text())
  .then(htmlGenerato => {
    // Sostituisce il placeholder con la lista reale
    listActivities.innerHTML = htmlGenerato;
  })
  .catch(error => {
    console.error("Errore nel caricamento delle attività:", error);
    listActivities.innerHTML = '<li class="muted">Errore nel caricamento delle attività.</li>';
  });
</script>
```



Futuri aggiornamenti possibili



Problemi Critici

SQL Injection — Le query usano variabili inline (WHERE username = '\$username'). Bisogna passare ai **prepared statements** con bind_param(). Questo riguarda quasi tutti i file PHP.

Password admin hardcoded in chiaro — login.php riga 28 confronta direttamente \$password === 'admin'. L'admin dovrebbe stare nel DB come gli altri utenti.

access_password varchar(10) nel database — bcrypt genera hash di ~60 caratteri. La colonna tronca silenziosamente l'hash, rompendo l'autenticazione. Deve essere varchar(255).

Pagine HTML senza protezione — dashboardStudiante.html è accessibile da qualunque utente non loggato semplicemente digitando l'URL. Il guard session è in .php ma la pagina effettiva è .html.



Architettura

Header/Footer via <iframe> — Pattern non standard. Causa problemi di layout (altezze fisse), performance (richiesta HTTP extra) e accessibilità. Meglio usare `<?php include 'header.php'; ?>`.

Typo nel codice: \$congnome in
registrazioneStudiante.php — bug silenzioso.



UI/UX

Nessuna progress bar per le ore FSL, nonostante `n_ore_fatte` e `n_ore_fsl` esistano già nel DB.

alert() del browser per i feedback — da sostituire con notifiche inline/toast.

Titolo sbagliato in `inserimentoAttivita.html`: dice "Dashboard Tutor" invece di "Inserimento Attività".



GRAZIE PER L'ATTENZIONE